

Class 7: Machine Learning 1

David Tin (PID: A17785446)

Table of contents

Background	1
K-means clustering	1
Hierarchical Clustering	7
Principal Component Analysis (PCA)	9
Analysis of UK food data	9
Data Import	9
Tidy the data	10
Exploratory analysis	11
PCA to the rescue	16
Digging Deeper (variable loadings)	24

Background

Today we will explore some core machine learning methods that are very popular in bioinformatics. These include **clustering** and **dimensionality reduction**.

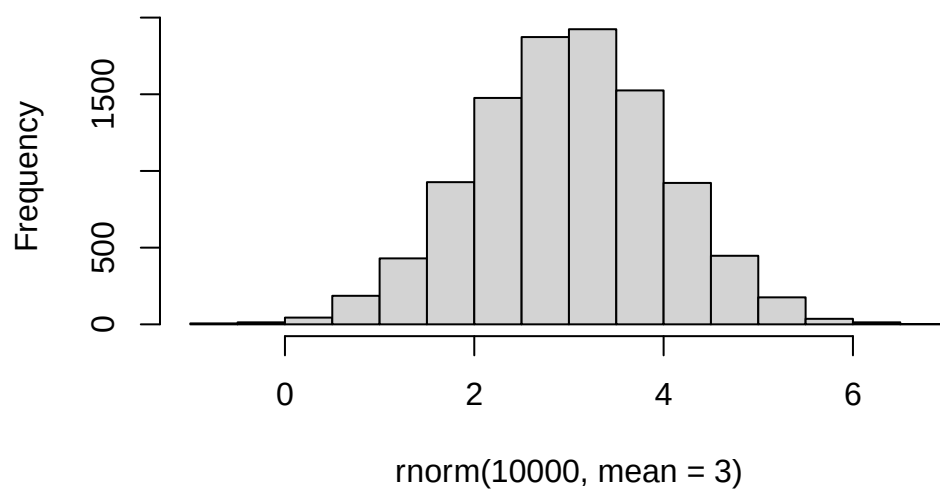
K-means clustering

The main function in “base” R for K-means clustering is called `kmeans()`

Before we go too deep let’s make up some “simple” data that we can cluster and know if we are getting a good answer or not. To help us do this we can use the `rnorm()` function:

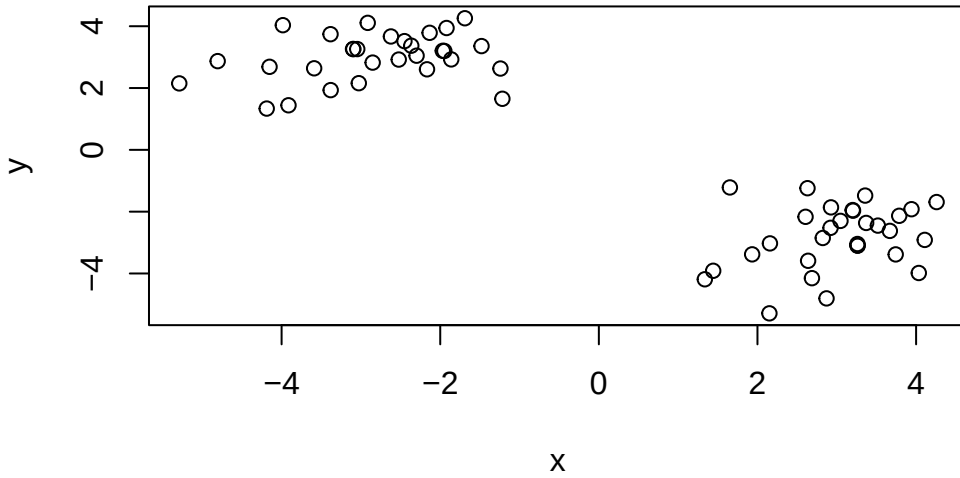
```
hist(rnorm(10000, mean=3))
```

Histogram of rnorm(10000, mean = 3)



```
x <- c(rnorm(30,-3), rnorm(30,+3))
```

```
z <- cbind(x=x, y=rev(x))  
plot(z)
```



Nowe we can run `kmeans()` on this input `z` amd see what the results look like.

```
km <- kmeans(z, centers = 2)
km
```

K-means clustering with 2 clusters of sizes 30, 30

Cluster means:

	x	y
1	-2.820139	2.993333
2	2.993333	-2.820139

Clustering vector:

[illegible]

Within cluster sum of squares by cluster:

```
[1] 48.0946 48.0946
(between_SS / total_SS = 91.3 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
```

```
attributes(km)
```

Q. How many points are in each cluster?

```
km$size
```

Q. What “component of your result object details cluster assignment/membership?”

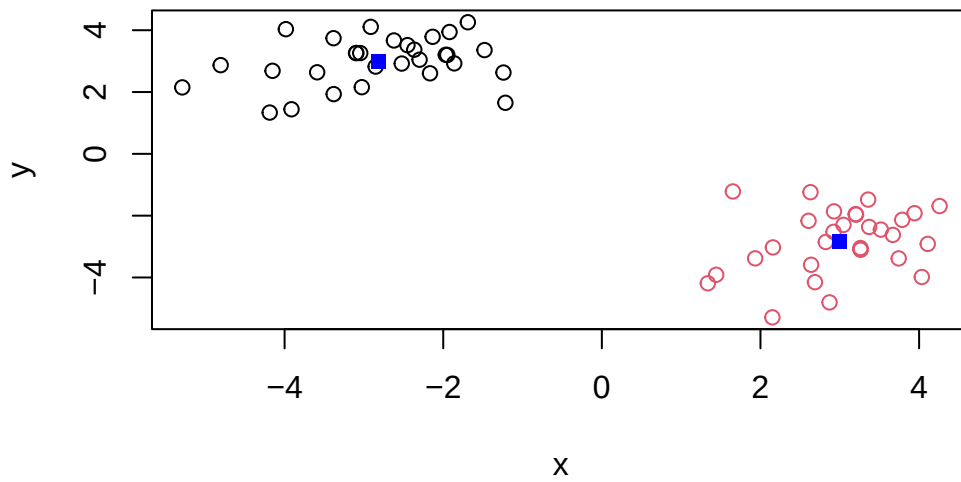
```
km$cluster
```

Q. What “componenet of your result object details cluster center?

km\$centers

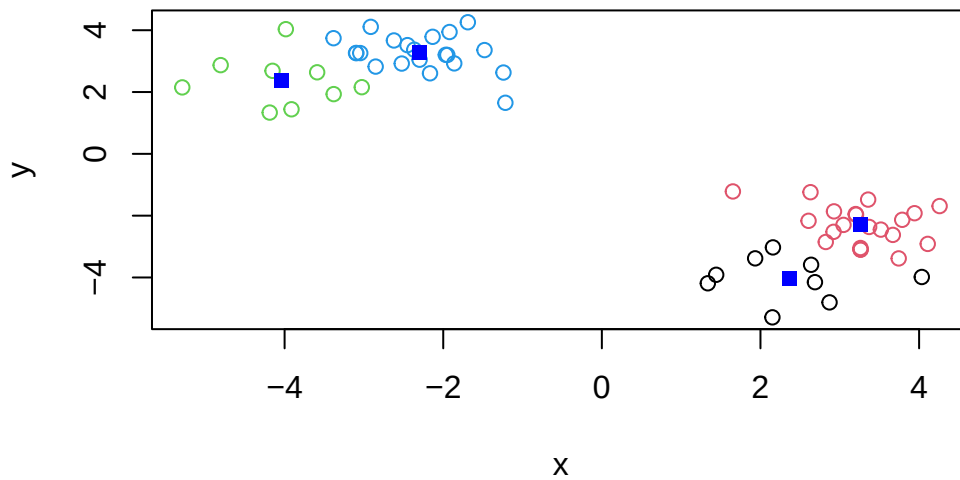
Q. Plot $\mathbf{z}$ colored by the kmeans cluster assignment and add cluster centers as blue points

```
plot(z, col=km$cluster)
points(km$centers, col="blue", pch=15)
```



Q. Run a K-means clustering and plot the results asking for 4 clusters (K=4)?

```
km4 <- kmeans(z, 4)
plot(z, col=km4$cluster)
points(km4$centers, col="blue", pch=15)
```

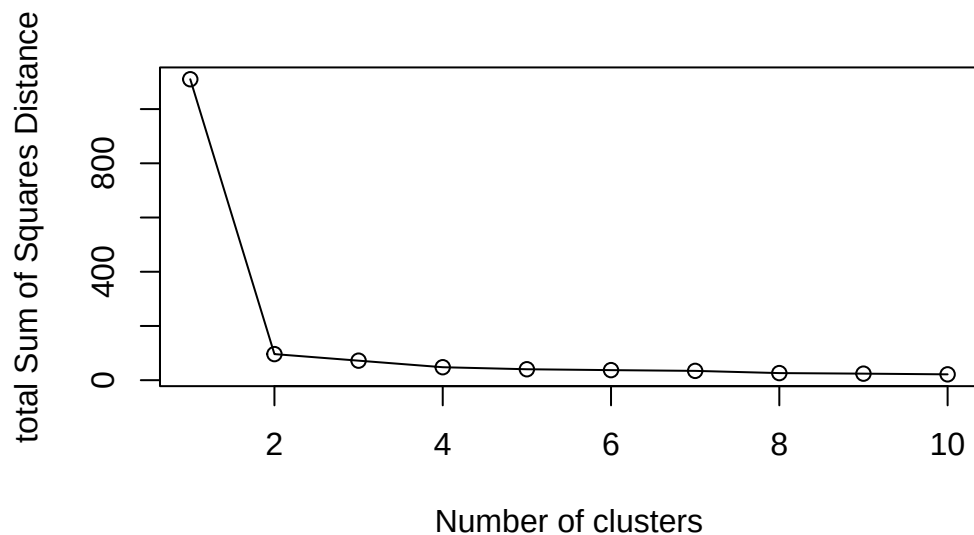


N.B. You need to tell K-means the number of clusters (i.e. set `centers=2`)!!

One approach is to try different values for `centers` and then pick the best.

```
ans <- NULL
for(i in 1:10){
  km <- kmeans(z, centers=i)
  ans <- c(ans, km$tot.withinss)
}

plot(ans, typ="o",
      xlab="Number of clusters",
      ylab="total Sum of Squares Distance")
```



Hierarchical Clustering

The main function in “base” R for Hierarchical Clustering is called `hclust()`

This function does not take your “raw” data for clustering. You must first build a “distance matrix” from your data and pass this as an input to `hclust()`

```
d <- dist(z)
hc <- hclust(d)
hc
```

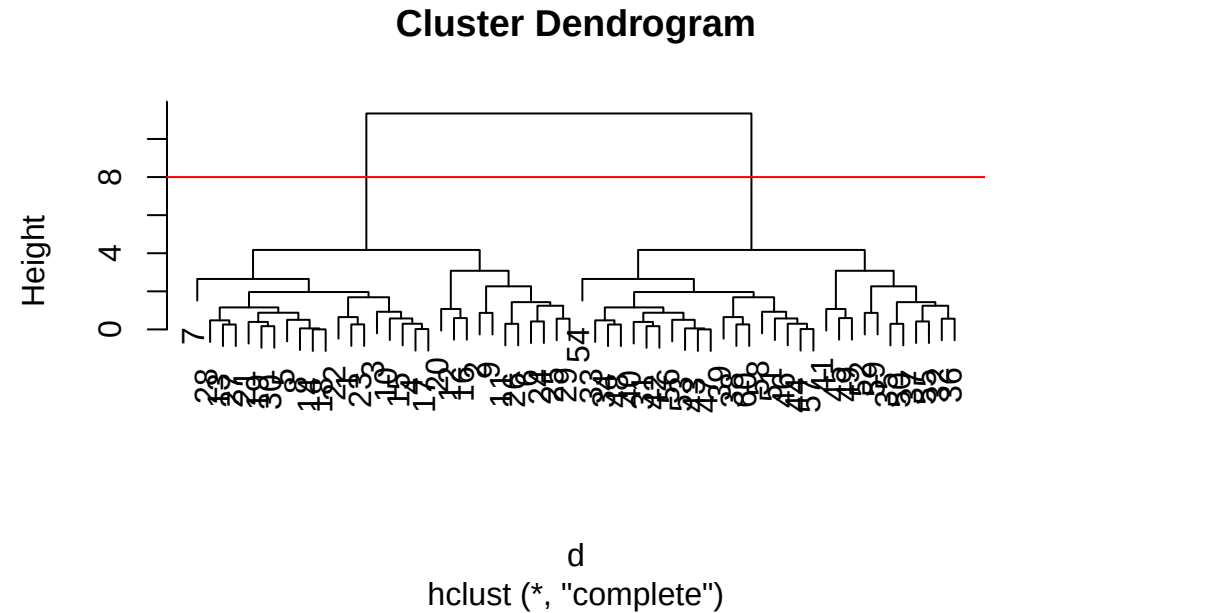
Call:

```
hclust(d = d)
```

```
Cluster method   : complete
Distance         : euclidean
Number of objects: 60
```

There is a bespoke `plot()` method for `hclust()` result objects.

```
plot(hc)
abline(h=8, col="red")
```



Once we have our `hclust` object (our “tree” of “cluster dendrogram”) we can “*cut*” the tree to reveal the clustering pattern.

```
cutree(hc, h=8)
```

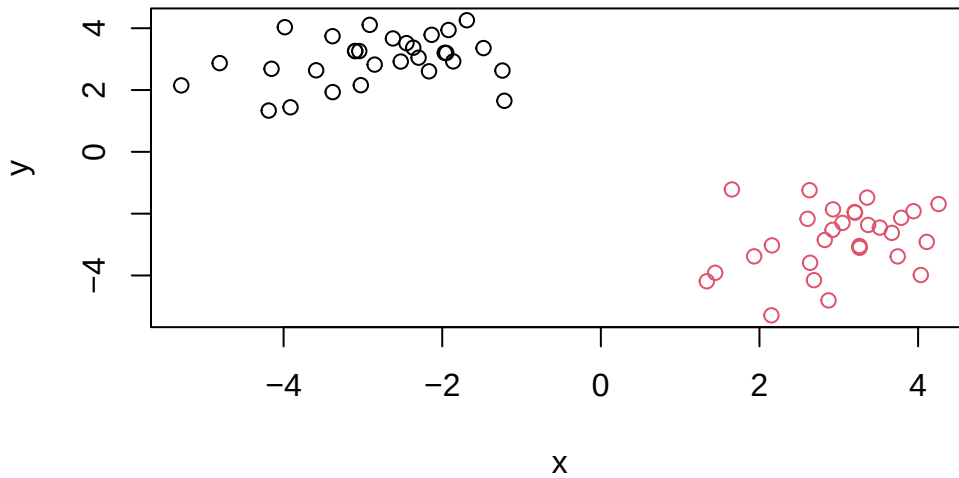
[illegible]

```
cutree(hc, k=4)
```

[1] 1 2 1 1 1 2 1 1 2 1 2 2 1 1 1 2 1 1 1 2 1 1 1 2 2 1 1 2 1 3 4 3 3 4 4 4
[39] 3 3 4 3 3 3 4 3 3 3 4 4 3 4 3 3 4 3 3 3 4 3

Q. Make a plot of $\mathbf{z}$ with your hclust results (i.e. colored by cluster membership)

```
grps <- cutree(hc,k=2)
plot(z, col=grps)
```

Principal Component Analysis (PCA)

PCA is a dimensionality reduction method that is popular for revealing patterns in complex datasets

Analysis of UK food data

Let's look at some data on the eating habits of folks from the UK to see if there are patterns and trends that have some regions being distinct from others.

Data Import

This data is made available in CSV format so we can use the `read.csv()` function for import to R:

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
head(x)
```

	X	England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66
2	Carcass_meat	245	227	242	267
3	Other_meat	685	803	750	586
4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139

Q1. How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

```
dim(x)
```

```
[1] 17 5
```

Tidy the data

Fix anything that went wrong with data import.

```
rownames(x) <- x[,1]
x <- x[,-1]
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

```
dim(x)
```

```
[1] 17 4
```

```
x <- read.csv(url, row.names=1)
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

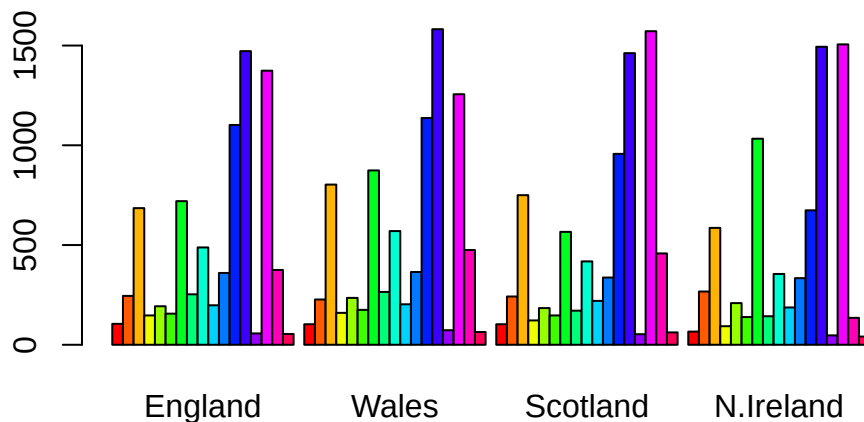
Q2. Which approach to solving the ‘row-names problem’ mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

I prefer the approach that sets the row names from the csv, rather than the minus indexing, as if the code with minus indexing is repeated, it keeps taking away columns. Repeating the `read.csv()` code will not take away columns and keep the data how we want it.

Exploratory analysis

Make some plots to help make sense of obvious trends.

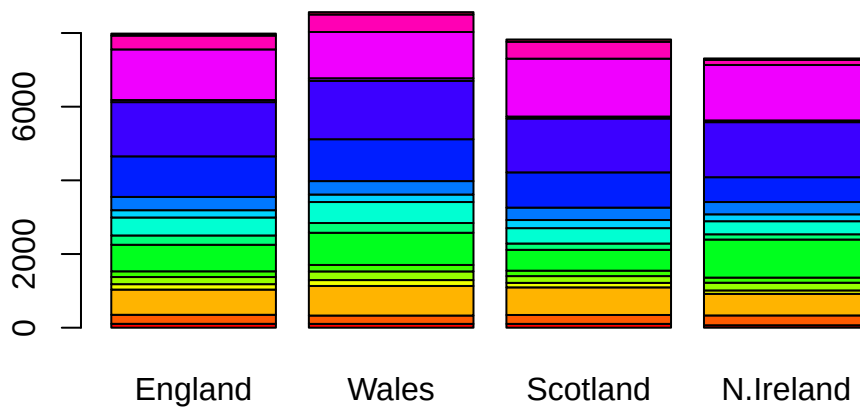
```
# Using base R
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```



Q3: Changing what optional argument in the above `barplot()` function results in the following plot (stacked bar chart)?

Changing the `beside` argument will result in a stacked barchart instead of putting the bars side by side.

```
# Using base R
barplot(as.matrix(x), beside=F, col=rainbow(nrow(x)))
```



```
library(tidyr)

# Convert data to long format for ggplot with `pivot_longer()`
x_long <- x |>
  tibble::rownames_to_column("Food") |>
  pivot_longer(cols = -Food,
               names_to = "Country",
               values_to = "Consumption")

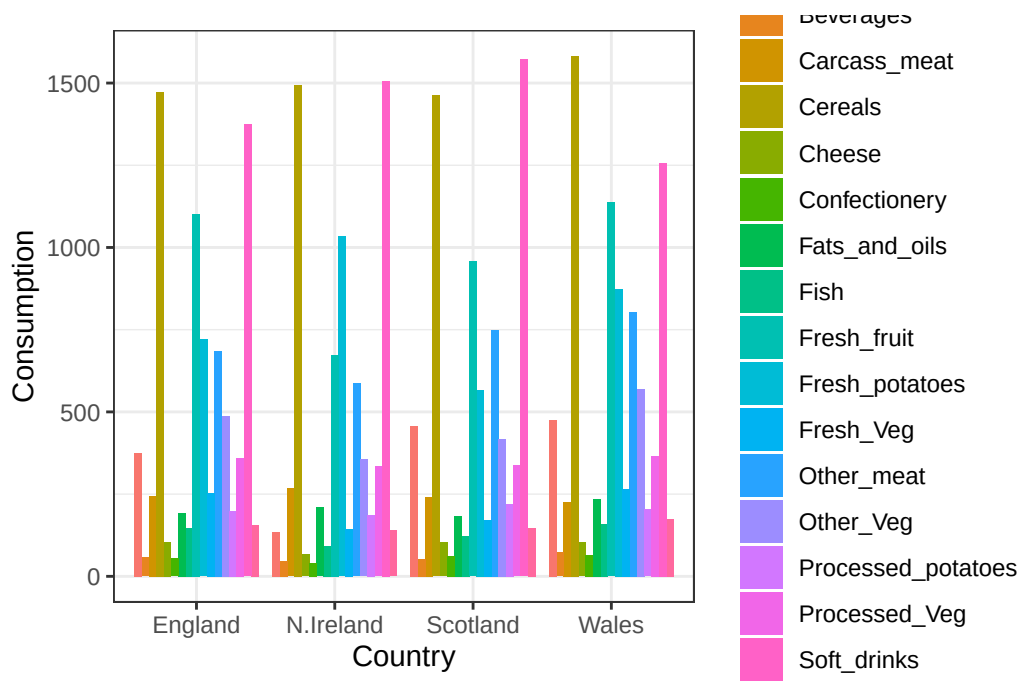
dim(x_long)
```

```
[1] 68  3
```

```
head(x_long)
```

```
# A tibble: 6 x 3
  Food      Country Consumption
  <chr>    <chr>      <int>
1 "Cheese"  England      105
2 "Cheese"  Wales        103
3 "Cheese"  Scotland     103
4 "Cheese"  N.Ireland     66
5 "Carcass_meat " England      245
6 "Carcass_meat " Wales        227
```

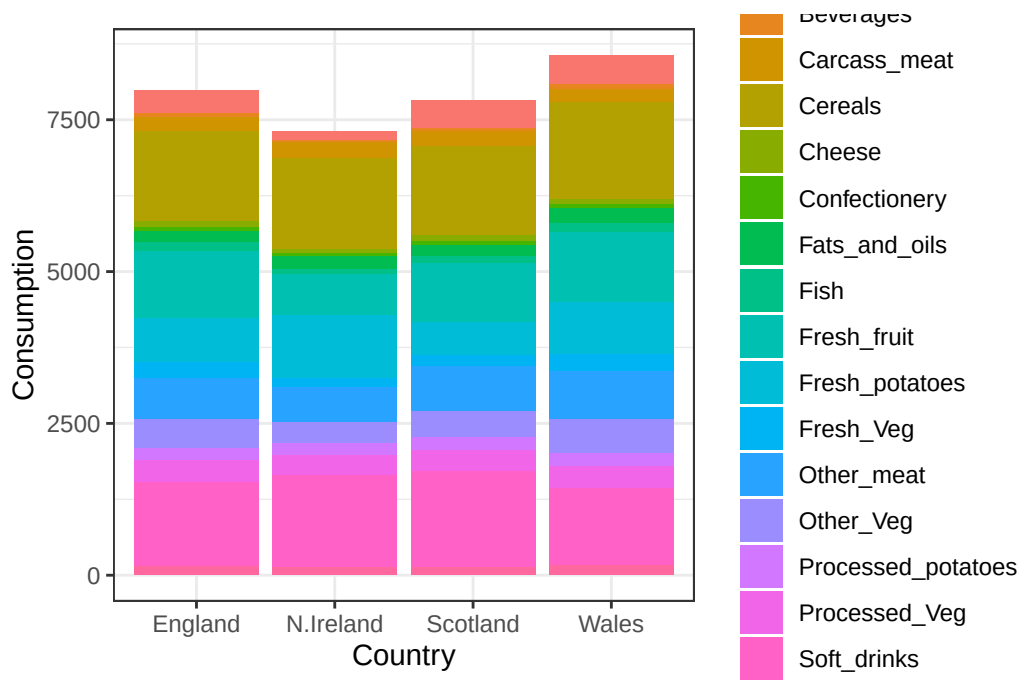
```
# Create grouped bar plot
library(ggplot2)
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "dodge") +
  theme_bw()
```



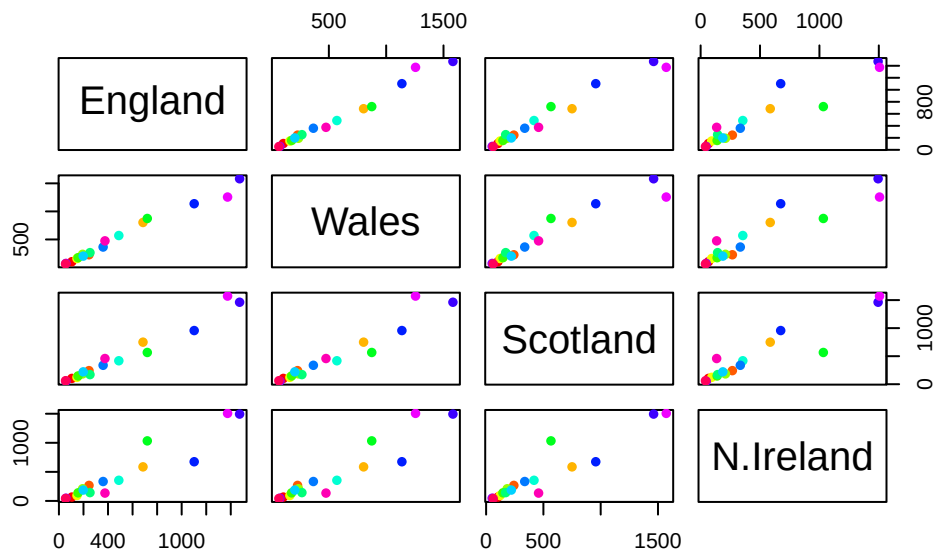
Q4: Changing what optional argument in the above `ggplot()` code results in a stacked barplot figure?

Deleting the argument `position = "dodge"` will result in a stacked barplot.

```
ggplot(x_long) +  
  aes(x = Country, y = Consumption, fill = Food) +  
  geom_col() +  
  theme_bw()
```



```
pairs(x, col=rainbow(nrow(x)), pch=16)
```

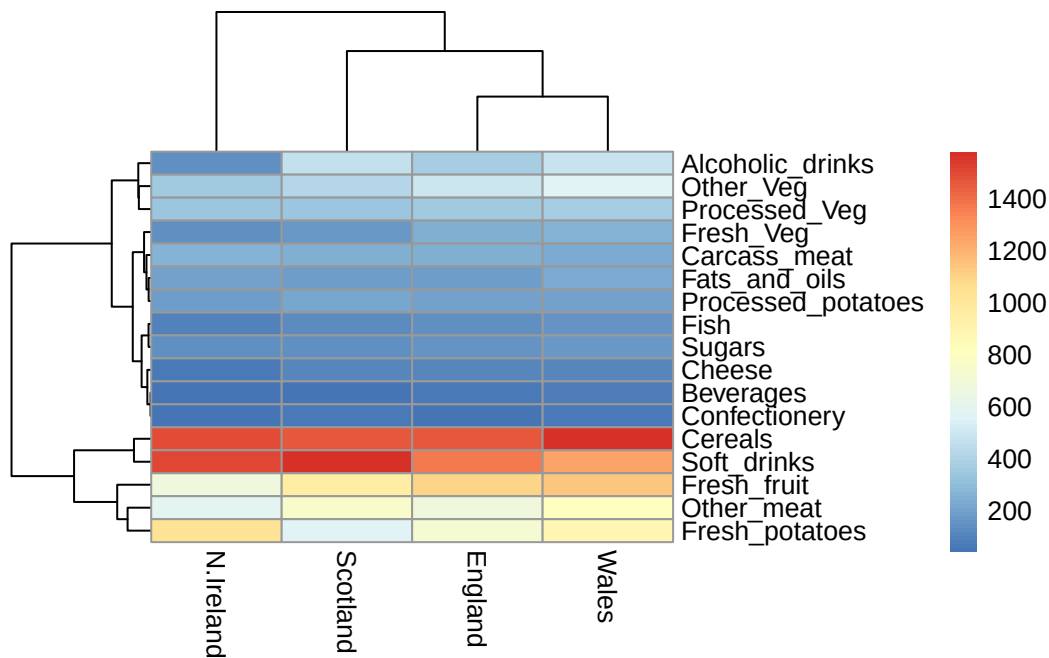


Q5: We can use the `pairs()` function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

If a given point lies on the diagonal for a given plot, it means that the two countries being compared consume a similar amount of that specific food type.

```
library(pheatmap)

pheatmap( as.matrix(x) )
```



Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

Key-point Even relatively small datasets can prove challenging to interpret

PCA to the rescue

The main function in “base” R for PCA is called `prcomp()`. This function wants the “observations” to be rows and the “variables” to be columns.

So here we need to take the transpose of our `x` input object

```
pca <- prcomp(t(x))
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	3.176e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

The returned `pca` object has components that we can use to make our main result figures:

```
attributes(pca)
```

```
$names
[1] "sdev"      "rotation" "center"    "scale"     "x"

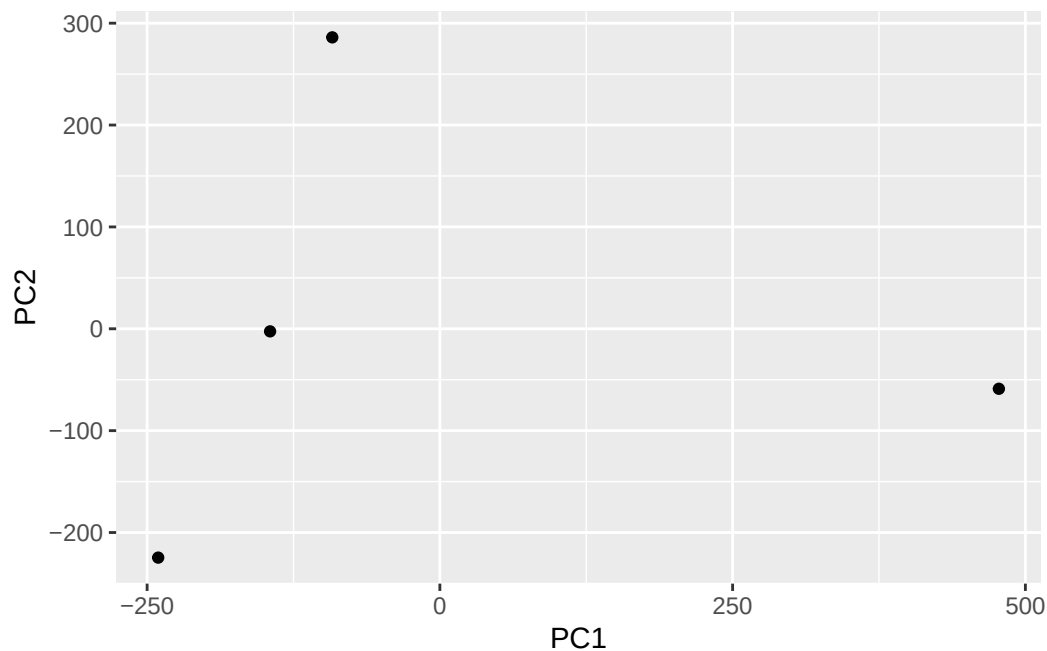
$class
[1] "prcomp"
```

The main result figure from this analysis is called a “PC score plot” or “ordination plot”, “PC plot” or “PC1 vs PC2” plot.

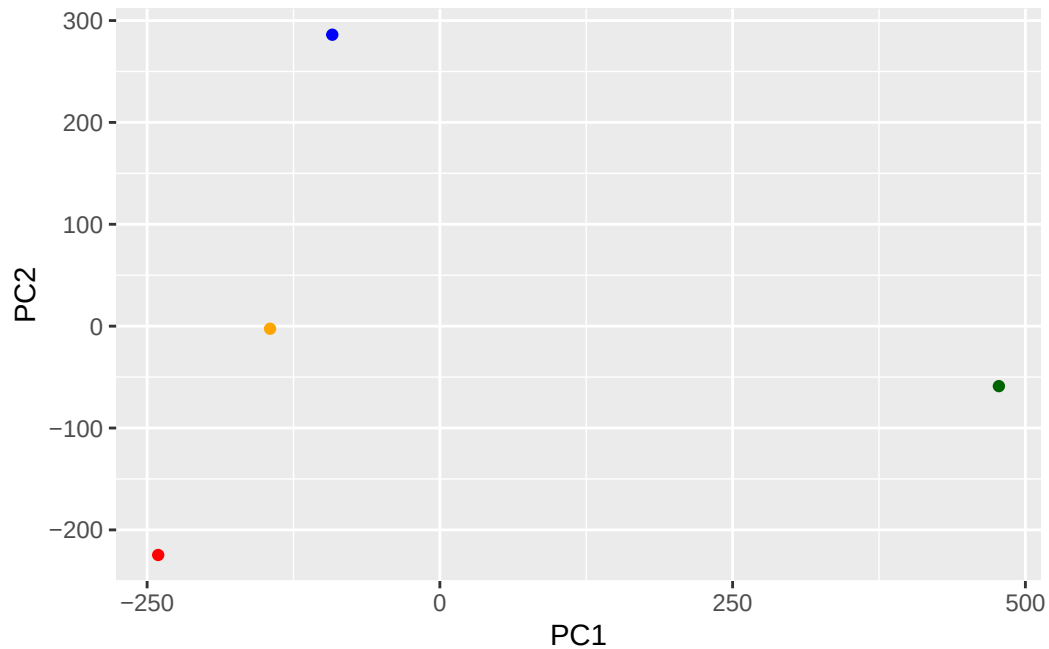
This plot shows how samples (in this case counties) relate to each other along our new PC axis

This is our new “reduced-dimensional space”. In this case 2 dimensions, PC1 and PC2, that capture most of the variance in the original 17 dimensional data-set.

```
library(ggplot2)
ggplot(pca$x) +
  aes(PC1,PC2)+
  geom_point()
```



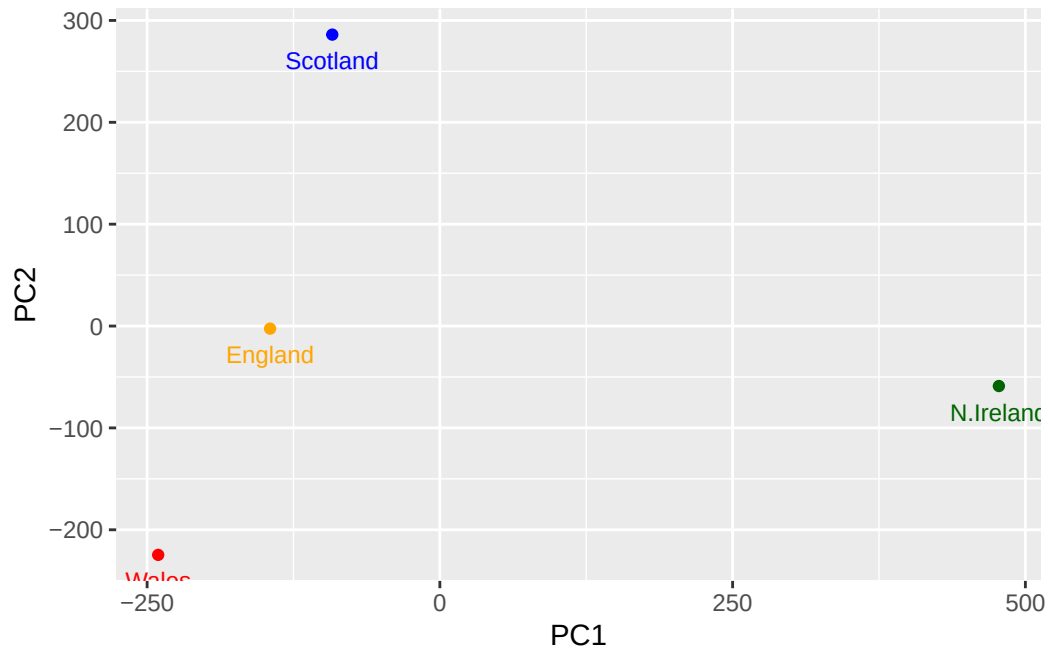
```
mycols <- c("orange","red","blue","darkgreen")
ggplot(pca$x) +
  aes(PC1,PC2)+
  geom_point(col=mycols)
```



```
pca$x
```

	PC1	PC2	PC3	PC4
England	-144.99315	-2.532999	105.768945	-4.894696e-14
Wales	-240.52915	-224.646925	-56.475555	5.700024e-13
Scotland	-91.86934	286.081786	-44.415495	-7.460785e-13
N.Ireland	477.39164	-58.901862	-4.877895	2.321303e-13

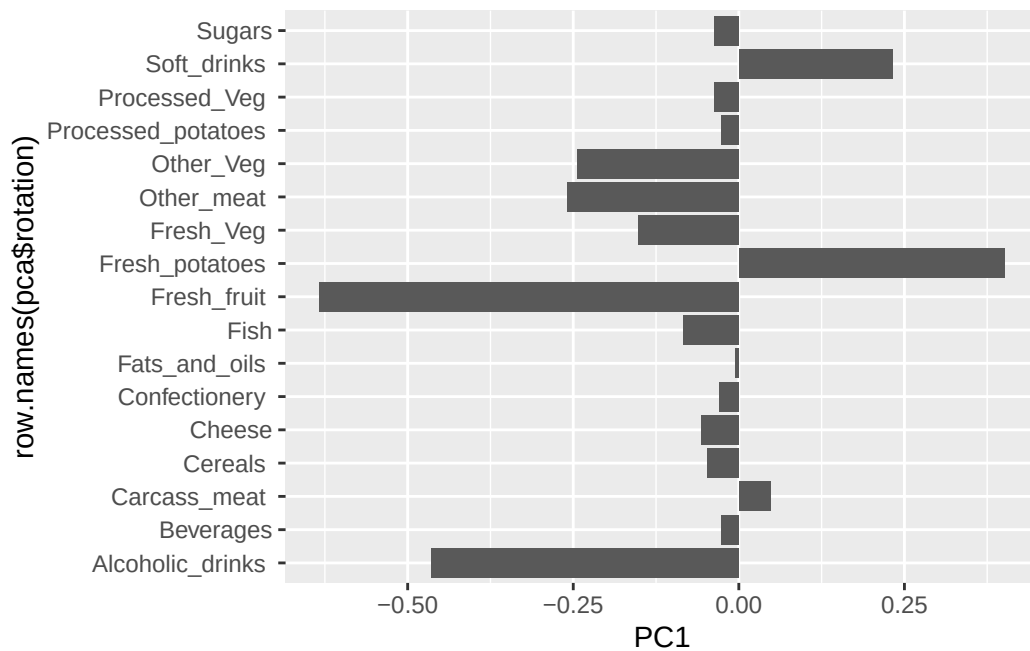
```
ggplot(pca$x) +
  aes(PC1,PC2, label=row.names(pca$x))+
  geom_point(col=mycols) +
  geom_text(size=3, vjust=2, col=mycols)
```



```
pca$x
```

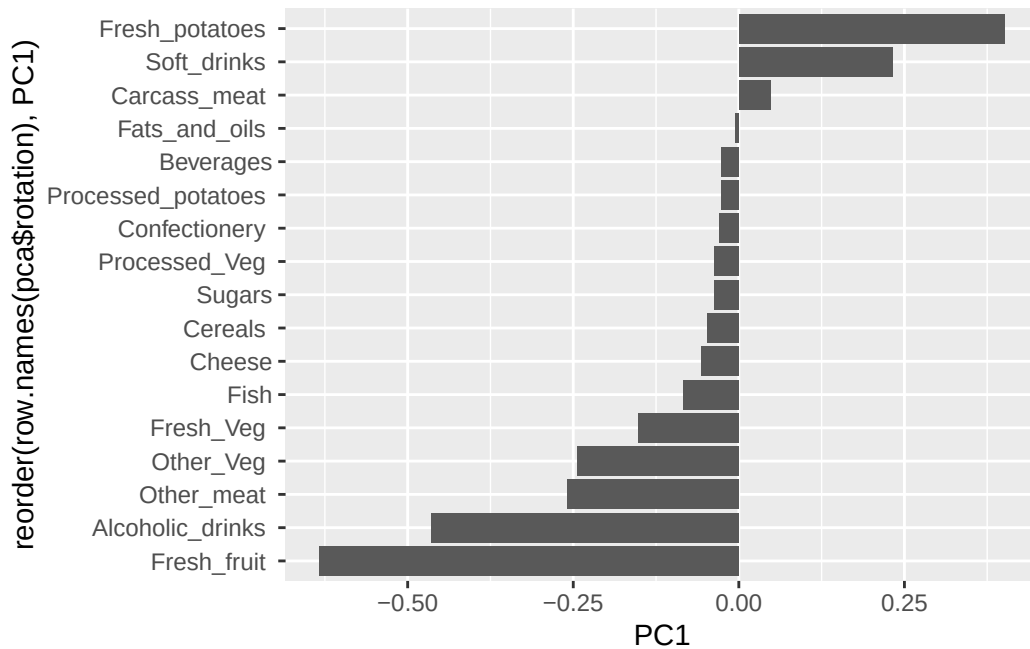
	PC1	PC2	PC3	PC4
England	-144.99315	-2.532999	105.768945	-4.894696e-14
Wales	-240.52915	-224.646925	-56.475555	5.700024e-13
Scotland	-91.86934	286.081786	-44.415495	-7.460785e-13
N.Ireland	477.39164	-58.901862	-4.877895	2.321303e-13

```
ggplot(pca$rotation) +
  aes(PC1, row.names(pca$rotation)) +
  geom_col()
```



PC score plots show the similarities and patterns between groups, where loading plots show the contribution to which the groups have on PC.

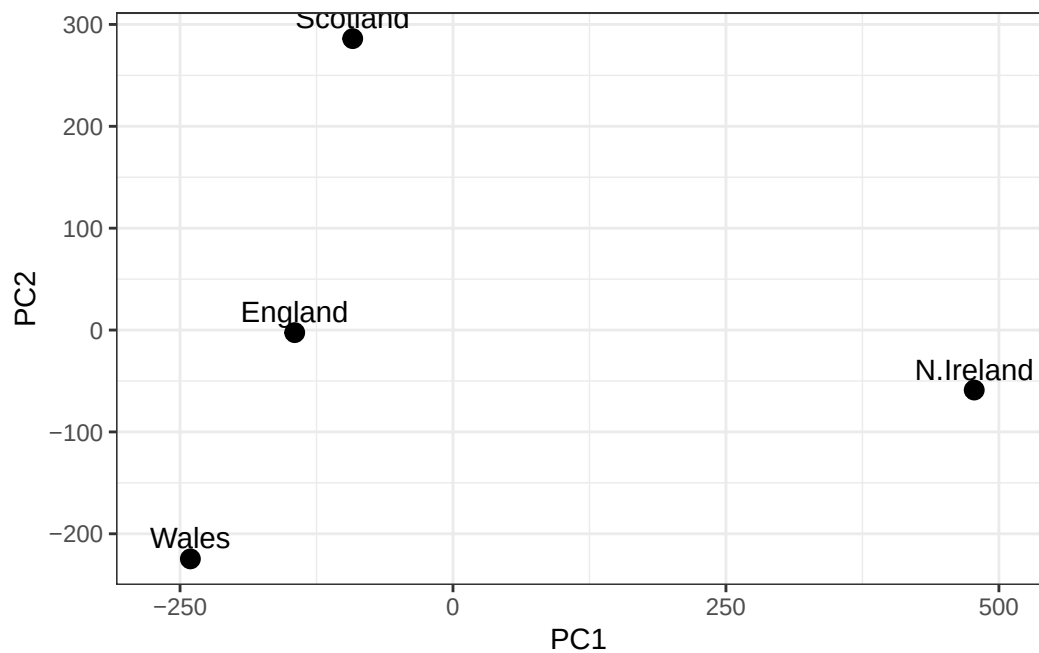
```
ggplot(pca$rotation)+
  aes(PC1,
       reorder(row.names(pca$rotation), PC1)) +
  geom_col()
```



Q7. Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

```
# Create a data frame for plotting
df <- as.data.frame(pca$x)
df$Country <- rownames(df)

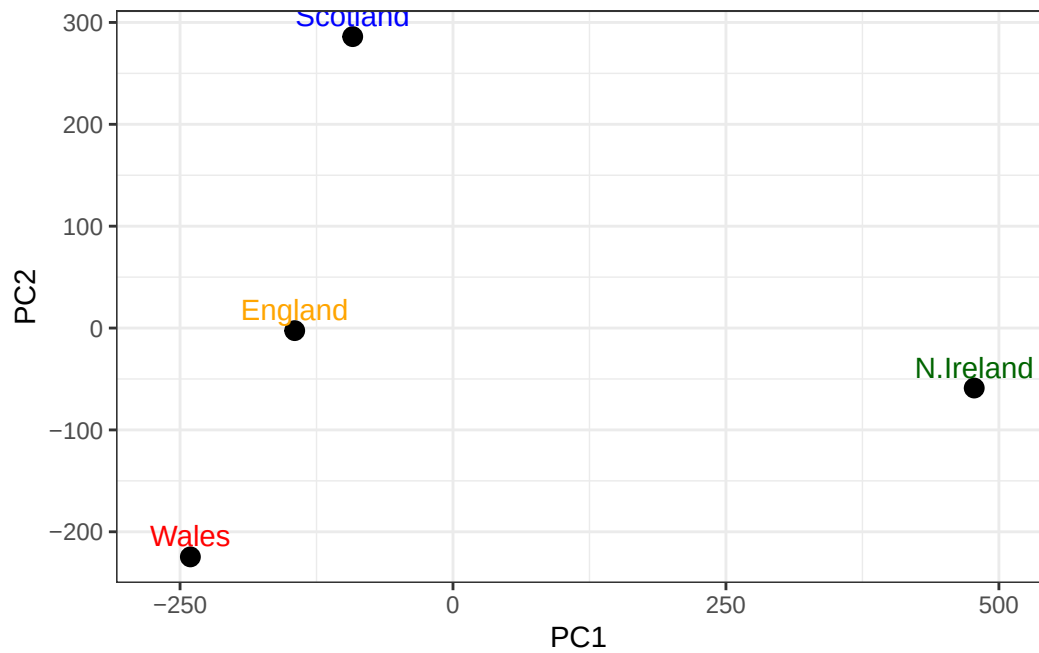
# Plot PC1 vs PC2 with ggplot
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



Q8. Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.

```
# Create a data frame for plotting
df <- as.data.frame(pca$x)
df$Country <- rownames(df)

# Plot PC1 vs PC2 with ggplot
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5, col=mycols) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```

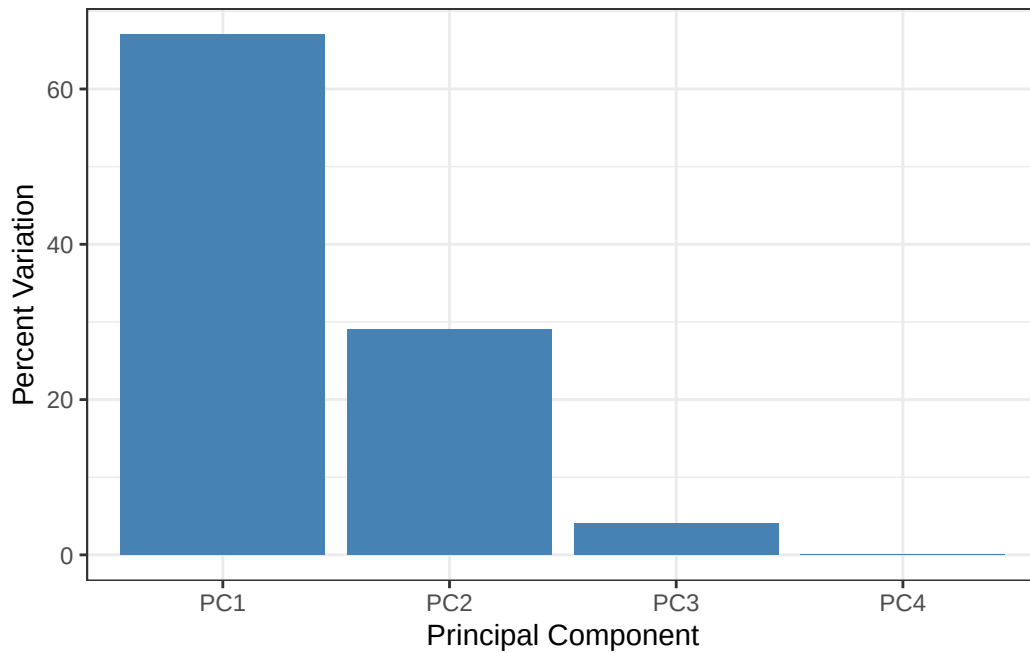


```
v <- round( pca$sdev^2/sum(pca$sdev^2) * 100 )
v
```

```
[1] 67 29 4 0
```

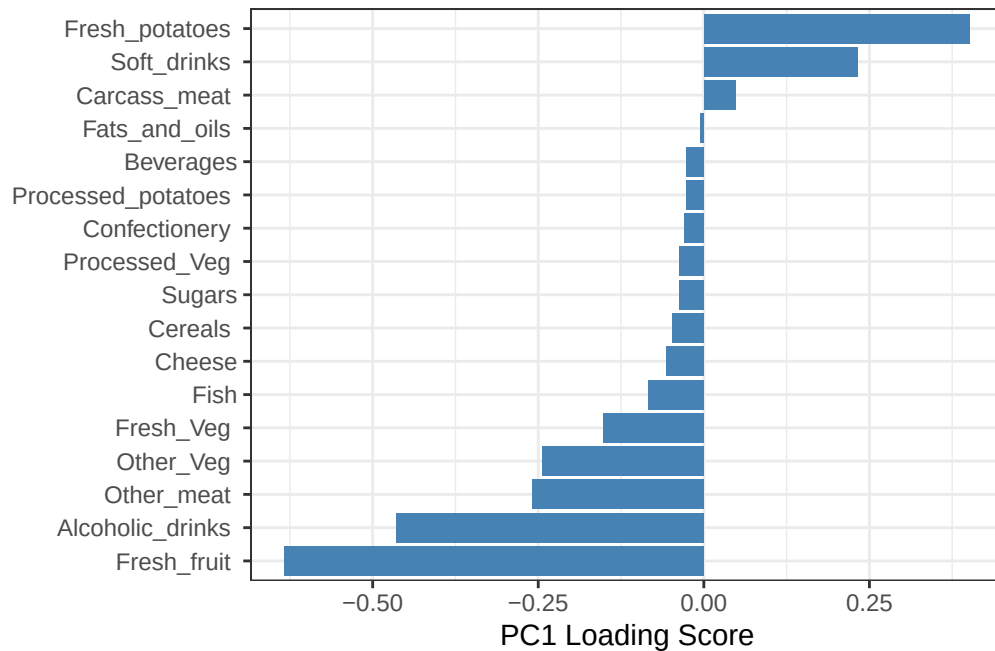
```
# Create scree plot with ggplot
variance_df <- data.frame(
  PC = factor(paste0("PC", 1:length(v)), levels = paste0("PC", 1:length(v))),
  Variance = v
)

ggplot(variance_df) +
  aes(x = PC, y = Variance) +
  geom_col(fill = "steelblue") +
  xlab("Principal Component") +
  ylab("Percent Variation") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 0))
```



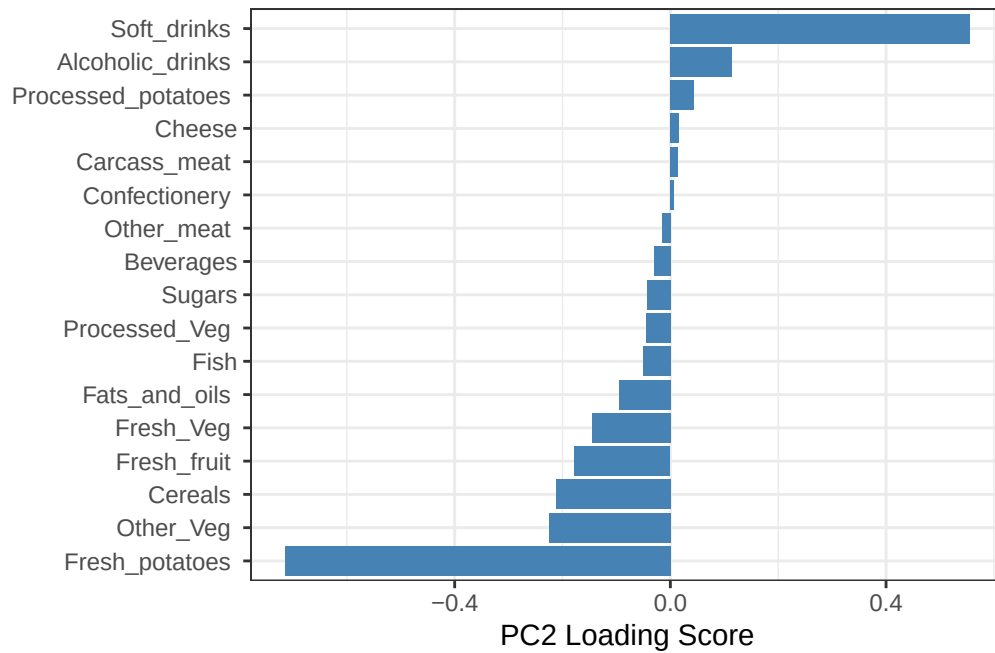
Digging Deeper (variable loadings)

```
## Lets focus on PC1 as it accounts for > 90% of variance
ggplot(pca$rotation) +
  aes(x = PC1,
      y = reorder(rownames(pca$rotation), PC1)) +
  geom_col(fill = "steelblue") +
  xlab("PC1 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```

Q9: Generate a similar 'loadings plot' for PC2. What two food groups feature prominently and what does PC2 mainly tell us about?

```
ggplot(pca$rotation) +
  aes(x = PC2,
      y = reorder(rownames(pca$rotation), PC2)) +
  geom_col(fill = "steelblue") +
  xlab("PC2 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



The loading plot for PC2 shows that fresh potatoes push northern Ireland down (-y), as fresh potatoes have a large negative loading score on PC2 which is the y-axis on the PC plot. However, soft drinks push Scotland up, having a large positive loading score.